Phase 2 — pgvector RAG (Vector Search + Hybrid Retrieval)

Thesis priority: #1 (RAG retrieval quality) from §5.5 Future Works. **Date:** 2026-10-05 (vector search) / 2026-10-06 (hybrid retrieval) — FYP 2, Week 1 **Status:** complete — vector search, hybrid rescoring, and widened news-agent window all implemented and evaluated against the production Supabase corpus (1,692 embedded articles).

---

1. Problem statement (from FYP 1 evaluation, thesis §5.4)

The current retrieval is in-memory, uses Ollama embeddings, and searches a limited candidate pool. It can miss context needed for complex questions.

Concrete FYP 1 weaknesses, as found in `src/ai/retriever.py`:

1. **Re-embedding everything per question** — every candidate article's text was embedded on every user query (N Ollama calls per question; the candidate pool was capped at 15 partly for this reason).
2. **No persistence** — the embedding cache lived in process memory; every bot restart reset it, and each restart paid the embedding cost again.
3. **Pool-size ceiling** — ranking quality was limited to the 15 most recent candidate articles selected by recency/category rules before semantics ever entered the picture.

2. Changes made

2.1 Schema (Supabase Postgres)

- Enabled the `pgvector` extension (v0.8.0) on the Supabase project.
- New table `article_embeddings` (migration: `migrate_create_article_embeddings_table`, idempotent, Postgres-only):
- `article_id INTEGER UNIQUE REFERENCES news_articles(id)`
- `embedding vector(768) NOT NULL` (dimension follows `RAG_EMBEDDING_DIM`; must match `OLLAMA_EMBED_MODEL` — nomic-embed-text = 768)
- `model VARCHAR(128)` — embed-model provenance; also lets future model swaps detect stale rows
- HNSW index `ix_article_embeddings_hnsw` with `vector_cosine_ops` for fast cosine top-k.

SQLite deployments are unaffected: the model/table is excluded from `create_all` and the migration is dialect-guarded (backward compatibility per change rule #2).

2.2 Config (`src/core/config.py`)

Variable	Default	Purpose
`RAG_VECTOR_ENABLED`	`true`	Master switch; `false` restores exact FYP 1 behavior.
`RAG_EMBEDDING_DIM`	`768`	Vector width; must match the embed model.
`RAG_EMBED_BATCH_SIZE`	`8`	Backfill batch size.

2.3 Retriever (`src/ai/retriever.py`)

- New `_vector_top_articles()`: embeds **only the user query**, then runs `ORDER BY embedding <=> $qv LIMIT k` (pgvector cosine distance) restricted to the candidate pool's article IDs — preserving FYP 1's pool semantics (recency/category pre-filter, semantics re-rank).
- `semantic_rank_articles()` now tries the vector path first and **falls back to the exact FYP 1 in-memory path** when: the feature is off, the DB is SQLite, the table is missing, the query embedding fails (Ollama down), or the SQL errors. Degradation contract from AGENTS.md rule #8 is preserved.

2.4 Keeping vectors fresh

- `services.embed_articles_for_ids()` — embeds any article missing a vector row; called at the end of `backfill_ai_summaries_for_article_ids()` so newly ingested articles get vectors right after their AI summary (guarded, best-effort, never breaks prefetch).

- `scripts/backfill_embeddings.py` — resumable one-shot backfill over existing `news_articles` (`--limit` for smoke tests, `--counts` for coverage, `--dry-run`).
- `cleanup_service.cleanup_old_news_data()` — now deletes embedding rows **before** deleting old articles (FK has no cascade; without this the Postgres delete would fail). Also returns `embeddings_deleted` for logs.

3. Why pgvector (design rationale, for the report)

- **No new infrastructure**: the system already moved to Supabase Postgres in FYP 2 prep; pgvector turns the existing database into the vector store — no separate service (Qdrant, Pinecone, Chroma) to host, back up, or pay for. Fits the thesis's cost/privacy posture.
- **Persistence + shared state**: embeddings survive restarts and are shared by any bot instance (relevant to the Fly/Oracle deployment story).
- **SQL-native hybrid**: the candidate-pool pre-filter (recency, category, location) and the semantic re-rank now live in the same database; future work can push the entire pool selection into SQL.
- **Honest limitation**: embedding quality is still bounded by the local embedding model (`nomic-embed-text`); pgvector improves *where* vectors live and *how* they're searched, not their semantic richness.

4. Evaluation (2026-10-05, production Supabase corpus: 1,692 articles, all embedded)

4.1 Equivalence check — PASSED

With the candidate pool fully embedded, the vector path and the FYP 1 in-memory path produce **identical rankings on 10/10 test questions** (`benchmark/rag_test_20261005T150329Z.json`). This is the expected correctness property: stored vectors are the same nomic-embed-text embeddings FYP 1 computed on the fly, so results must match. It also empirically confirms the graceful-fallback chain (an earlier run with an unembedded pool silently fell back on all questions — correct, but worth knowing fallback can mask the vector path in small tests).

4.2 Pool-coverage experiment — the headline result

The FYP 1 news agent only sees articles from the **last 24 hours** (`services.get_news_agent_response_for_user`, `cutoff = now - 24h`). For 10 questions with a known relevant article (`scripts/eval_rag_pools.py`, results in `benchmark/rag_pool_coverage.csv`):

Path	Reachable relevant articles
FYP1 (24h window)	**1–2 / 10** (run-dependent: the crime question's article was 27h old, straddling the boundary)
FYP2 (vector top-10 over the full 30-day corpus)	**3 / 10**

The vector path *can* reach every article (any pool size costs 1 query embedding + SQL), but pure embedding similarity does not reliably rank the relevant article top-10.

4.3 Diagnosis (measured, not assumed)

Question	Relevant article rank (of 1,692)	Cosine	Likely cause
helicopter crash Long Lellang	~107	0.722	broad topical match, beaten by 100+ similar articles
arson on 24h shops (Malay title, English query)	~394	0.556	cross-lingual gap
Bebuling Airport Betong	~982	0.575	rare proper noun carries no semantic weight for nomic-embed-text

Interventions tried:

- **nomic contrastive prefixes** (`search_query:` / `search_document:`) — implemented and fully re-backfilled (1,692 vectors, wiped and recomputed). Result: **no change (3/10)**. Recorded as a negative result; prefixes are kept anyway (model-intended usage, no cost).

4.4 Conclusion and next step (for the report)

The bottleneck is no longer *where* vectors live or *how many* articles can be searched — it is the **ranking signal** of a small embedding model over short RSS snippets. The system already extracts structured metadata per article (location, category); the next iteration (§4.5) is **hybrid retrieval**: pgvector cosine + a rescore boost for location/category/entity matches between question and article. The same eval script measures it.

Other honest limitations at this point: production Q&A still stubs to the 24h window in `get_news_agent_response_for_user`; widening that window is a one-line change once hybrid rescoring is in place (otherwise dilution, as seen with the crime question scoring in-24h but missing vector top-10).

4.5 Hybrid retrieval (2026-10-06) — implemented and evaluated

Not in the original FYP 2 plan (plan §5.5 lists vector search, dedup, official alerts, summaries). Added as an evidence-driven step: §4.3's diagnosis showed the embedding model is the ranking bottleneck, and the pipeline already extracts the metadata that could compensate. Approved by project owner 2026-10-06.

Design

Three additions on top of the pgvector path, all in `src/ai/retriever.py`:

1. **Metadata boosts** (`_hybrid_rescore`): after vector fetch, add to each article's cosine

• `RAG_HYBRID_W_KEYWORD` (0.25) × fraction of the question's distinctive tokens present in the article (rescues rare proper nouns like "Bebuling");

• `RAG_HYBRID_W_LOCATION` (0.15) when the question names a Sarawak location matching the article's extracted location;

• `RAG_HYBRID_W_CATEGORY` (0.10) when the question names the article's category.

Anywhere location scan (`_query_locations`): the pipeline extractor anchors locations at the start of a headline, but users name them mid-sentence ("…in Kuching?"), so the question is scanned directly against `SARAWAK_LOCATION_ALIASES`.

Metadata recall union (`_metadata_recall_ids`): articles with a location match, or sharing ≥ `RAG_HYBRID_RECALL_MIN_OVERLAP` (0.5) of the question's tokens, are **unioned into the rescored set** regardless of cosine. Without this, boosts are cosmetic: the first hybrid eval scored 3/10 — *identical* to vector-only, because every rescued miss sat at vector rank 107–982, outside the fetch-deep-50 window the rescore could reorder. Acronyms (SEZ, API, SESCO) are kept as tokens regardless of length — the ≥4-letter filter had silently dropped them.

All weights and switches are env-tunable in `src/core/config.py` (`RAG_HYBRID_ENABLED` defaults on; `false` restores pure vector ranking).

Result (same 10 questions, same corpus; `benchmark/rag_pool_hybrid_20261006.txt`)

Path	Relevant article in top-10
FYP1 (24h window)	1 / 10
FYP2 vector only	3 / 10
FYP2 hybrid	**6 / 10**

The three hybrid rescues: **Serian haze/API** (location + keyword), **Bebuling Airport** (keyword recall of a rare proper noun at vector rank ~982), **SEZ** (acronym keyword). The four remaining single-article misses are mostly **not** quality failures — the verbose top-10s show the right *story* surfaces: for "helicopter crash at Long Lellang" **all 10** results are about that crash (sibling follow-up articles outrank the designated one); for "arson on 24-hour shops" 10/10 are about those arson attacks, including the Malay-language article; for "crime news in Kuching" 10/10 are Kuching crime/police stories. The strict per-article metric undercounts answer quality; a same-story cluster metric is future work.

Process record (negative results kept for the report):

• v1 boost-only hybrid: 3/10 = no change. Root cause: rescoring reorders only what the vector fetch returns; the misses sat at ranks 107–982. → led to the recall union (v2).

• v2 recall union: 5/10. Debugging the SEZ miss showed "will" leaking into the query's distinctive tokens and diluting the keyword fraction (2/3 instead of 2/2). → added modal verbs to the stopword list (v3).

- v3: 6/10 (final).

Residual limitations (honest, for §5.4-style discussion)

- **Cross-lingual gap persists**: an English question against a Malay-titled article (arson case, cosine 0.556, vector rank ~394) is rescued into the pool only when its location matches; keyword boosts cannot fire cross-lingually.
- **Embedding ceiling**: when the relevant article's cosine is far below the top-10 boundary (SEZ was 0.48 vs ~0.67 before boosts), +0.25 boost barely closes the gap — it only just made top-10. A stronger embedding model or a cross-encoder reranker is the next lever if FYP 2 budget allows.
- **Sibling competition**: big stories generate many similar articles that split the keyword boost equally; ranking within a story cluster is still cosine-driven.

Production window change (same commit)

`get_news_agent_response_for_user` now uses `NEWS_AGENT_WINDOW_HOURS` (default 720h = the full 30-day retention corpus) instead of the hardcoded 24h, and the semantic pool grew from 15 to `RAG_AGENT_POOL_SIZE` (default 400). Without hybrid, widening the window would have diluted quality; with it, old-but-relevant stories become reachable (several eval relevant articles were 18–27 days old).

5. Reproduction

```
# extension + table (idempotent; runs automatically at bot startup via init_db)
python -c "from src.storage.database import init_db; init_db()"

# backfill (chunked smoke test first)
python scripts/backfill_embeddings.py --limit 20
python scripts/backfill_embeddings.py --counts
python scripts/backfill_embeddings.py # full run
```

6. Threats to validity / risks

1. Embedding dim is fixed at table creation; switching `OLLAMA_EMBED_MODEL` to a model with a different width requires `DELETE FROM article_embeddings` + re-backfill (documented here and in config).
2. Articles without embeddings silently don't surface in vector ranking until backfilled — acceptable because RAG still has its recency/category pre-filter and the in-memory fallback.
3. Retention cleanup deletes articles after 30 days, taking their embeddings with them; RAG therefore effectively covers recent news, which matches the product's news-Q&A scope.
4. Backfill inserts rows one commit per batch; an interrupted run is resumed by re-running (already-embedded articles are skipped).